Learn IREE:
A Beginner Course on MLIR-Based Compilation and Runtime

Djordje Todorovic

March 2026

Contents

1 Preface **4**
1.1 Who This Course Is For . . . 4
1.2 Prerequisites . . . 4
1.3 Version Note . . . 4

2 What IREE Is and Why It Exists **4**
2.1 Why People Use IREE . . . 5
2.2 What IREE Is Not . . . 5

3 The Big Picture **5**
3.1 End-to-End Pipeline . . . 5
3.2 A Visual Mental Model . . . 6
3.3 The Most Important Idea . . . 6

4 Core Vocabulary **6**

5 How IREE Relates to MLIR and xDSL **6**
5.1 MLIR, xDSL, and IREE in One View . . . 7
5.2 A Useful Mental Separation . . . 7
5.3 Why This Matters for Learning . . . 7

6 Compiler Stages You Should Recognize **7**
6.1 `flow` . . . 7
6.2 `stream` . . . 7
6.3 `hal` . . . 7
6.4 `vm` . . . 8

7 Setting Up IREE **8**
7.1 Build from Source . . . 8
7.1.1 Prerequisites . . . 8
7.1.2 Clone and Configure . . . 8
7.1.3 Build the Developer Tools . . . 8
7.1.4 Run the Test Suite . . . 9
7.2 Python Packages . . . 9

8 Meet the Main Tools **9**

9 How To Read an IREE Command Line **9**
9.1 Reading a Compile Command . . . 9
9.2 Reading a Run Command . . . 10
9.3 The Two Questions Beginners Mix Up . . . 10
9.4 A Tiny Backend Cheat Sheet . . . 10

10 Your First IREE Program **10**
10.1 Step 1: Write `simple_abs.mlir` . . . 11
10.2 Step 2: Compile It to `vmvx` . . . 11
10.3 Step 3: Run It . . . 11
10.4 What Just Happened . . . 11
10.5 Shortcut for Experimentation . . . 12

11 Running on CPU the Usual Way **12**
11.1 Compile for `llvm-cpu` . . . 12
11.2 Run It . . . 12
11.3 Check Available Drivers and Devices . . . 12
11.4 `local-sync` vs `local-task` . . . 13

12 Small Example 2: Scale a Vector **13**
12.1 Write `scale_by_two.mlir` . . . 13
12.2 Compile and Run . . . 13
12.3 Why This Example Matters . . . 13

13 How `iree-run-module` Inputs Work **14**
13.1 Practical Advice . . . 14

14 Learning the Pipeline by Dumping Phases **14**
14.1 Dump All Compilation Phases . . . 14
14.2 How To Read Them . . . 15
14.3 Why This Is So Valuable . . . 15

15 HAL for Beginners **15**
15.1 What HAL Is . . . 15
15.2 Compile-Time HAL . . . 15
15.3 Runtime HAL . . . 16
15.4 Target Backend vs Runtime Driver . . . 16

16 Runtime Building Blocks **16**
16.1 Driver . . . 16
16.2 Device . . . 16
16.3 Allocator and Buffers . . . 17
16.4 Command Buffer . . . 17
16.5 Executable . . . 17
16.6 Semaphores and Fences . . . 17
16.7 Why This Section Matters . . . 17

17 VMFB and IREE Modules **17**
17.1 What a VMFB Is . . . 17
17.2 Inspect a Module . . . 18
17.3 A Useful Mental Model . . . 18

18 Small How-To 1: Benchmark a Module **18**

19 Small How-To 2: Dump Executable Files **18**

20 Small How-To 3: Isolate a Dispatch Benchmark **19**

21 Small How-To 4: Import an ONNX Model **19**

22 Debugging **20**
22.1 Use `local-sync` First 20
22.2 Debug `iree-run-module` with LLDB 20
22.3 GDB Alternative 20
22.4 Three Good Debug Habits 20

23 Troubleshooting Checklist **21**
23.1 A Good Investigation Order 21

24 Common Beginner Mistakes **21**

25 A Beginner Reading Order for the Source Tree **22**

26 Glossary **22**

27 Exercises **22**

28 Where To Go Next **23**

29 Official References **23**

30 Closing Note **23**

1 Preface

This course is a beginner-friendly introduction to IREE: the Intermediate Representation Execution Environment. It is written for engineers who already know some Python, C/C++, or ML tooling, but who are new to compiler pipelines and runtime systems.

The goal is not to turn you into an IREE core maintainer in one sitting. The goal is to give you a solid mental model, enough vocabulary to read the code and documentation, and enough hands-on examples to compile and run small programs without guessing.

1.1 Who This Course Is For

This material is a good fit if you are:

- an ML engineer who wants to understand what happens after model export,
- a compiler beginner who already knows what MLIR is at a high level,
- a systems engineer exploring runtimes, target backends, and hardware abstraction layers,
- a student who wants a practical entry point instead of a purely theoretical overview.

1.2 Prerequisites

You should be comfortable with:

- command-line work on Linux or macOS,
- basic programming concepts such as functions, types, and modules,
- the rough idea that machine learning programs operate on tensors.

You do **not** need to know the internals of LLVM, MLIR, or GPU drivers before starting.

1.3 Version Note

IREE changes quickly. The commands and package names in this course were aligned with the official IREE documentation available on March 9, 2026. If a command behaves differently on your checkout, first run `iree-compile -help` or `iree-run-module -help`, then compare with the current docs.

2 What IREE Is and Why It Exists

IREE is an MLIR-based compiler and runtime for deploying tensor programs across different kinds of hardware. The simplest mental model is:

> Take a model or MLIR program, lower it through several compiler stages, package the result into a deployable module, and execute it through a small runtime that talks to a real device.

That device may be:

- a CPU,
- a GPU through Vulkan, CUDA, HIP, or Metal,
- a reference backend such as `vmvx`,
- or a custom accelerator integrated through IREE's compiler and runtime extension points.

2.1 Why People Use IREE

IREE is useful because it gives you one compiler and one runtime framework for many deployment targets. That matters when you want:

- one toolchain across CPU, GPU, and accelerator targets,
- ahead-of-time compilation into deployable artifacts,
- small runtime footprints,
- clear separation between compilation and execution,
- a compiler pipeline built on top of MLIR instead of opaque framework internals.

2.2 What IREE Is Not

IREE is not:

- a training framework,
- a replacement for PyTorch, JAX, or TensorFlow authoring,
- a single monolithic backend with one hard-coded target.

It sits later in the stack. You author a program elsewhere, then hand it to IREE in a supported input form.

3 The Big Picture

3.1 End-to-End Pipeline

At a high level, the IREE flow looks like this:

```
Frameworks / frontends
  PyTorch, JAX, TensorFlow, ONNX, StableHLO, MLIR
            |
            v
        Import / export
            |
            v
        Input MLIR
            |
            v
  IREE compiler pipeline
    ABI -> preprocessing -> optimization
    -> flow -> stream -> hal -> vm
            |
            v
      VMFB deployable artifact
            |
            v
        IREE runtime
            |
            v
 HAL driver + executable loader + device
            |
            v
       Target hardware
```

Listing 1: High-level IREE pipeline

3.2 A Visual Mental Model

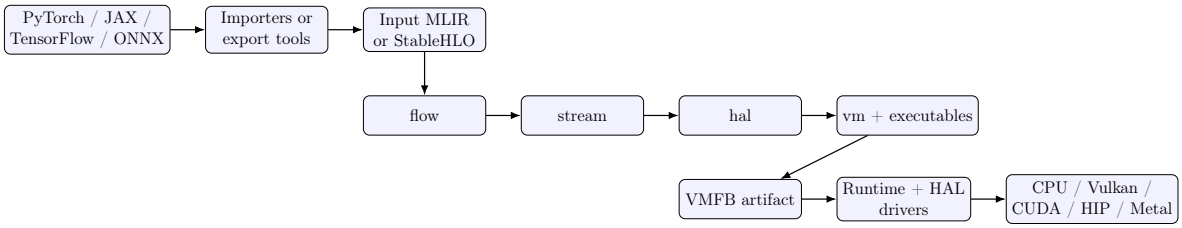

Figure 1: A beginner-friendly view of the IREE compiler and runtime stack.

3.3 The Most Important Idea

The most important idea in IREE is that **compilation and execution are separate concerns**.

- The **compiler** lowers high-level tensor programs into deployable artifacts.
- The **runtime** loads those artifacts, allocates buffers, dispatches work, and synchronizes with devices.

Once this clicks, many IREE terms become easier to place.

4 Core Vocabulary

MLIR Multi-Level Intermediate Representation. The compiler infrastructure IREE builds on.

Dialect A family of operations and types inside MLIR. Examples in IREE include `flow`, `stream`, `hal`, and `vm`.

Target backend The code generation backend selected at compile time, such as `llvm-cpu`, `vmvx`, `vulkan-spirv`, or `cuda`.

Runtime driver The runtime component that can actually execute compiled code on a device, such as `local-task`, `local-sync`, `vulkan`, or `cuda`.

VMFB A FlatBuffer file containing an IREE deployable module plus metadata and executable payloads.

Dispatch A unit of executable work handed to a device.

HAL Hardware Abstraction Layer. The layer that hides device-specific details behind a common interface.

5 How IREE Relates to MLIR and xDSL

This repository already contains xDSL material, so it helps to place IREE relative to both MLIR and xDSL.

5.1 MLIR, xDSL, and IREE in One View

- **MLIR** is the compiler infrastructure and IR framework.
- **xDSL** is a Python-native infrastructure inspired by MLIR and useful for learning, prototyping, and experimenting with compiler ideas.
- **IREE** is a production-oriented compiler and runtime stack built on MLIR for deploying tensor programs.

5.2 A Useful Mental Separation

xDSL mindset Build and understand compiler ideas quickly, often in Python, with explicit IR objects and passes you can inspect easily.

IREE mindset Take a real tensor program, lower it through a serious deployment pipeline, package it, and execute it on actual hardware through a runtime.

5.3 Why This Matters for Learning

If you already understand xDSL's representation-and-transformation model, you have a good base for reading IREE:

- both rely on explicit IR,
- both reward inspecting intermediate states,
- both make passes and dialect boundaries visible.

The difference is mainly **scope**. xDSL is excellent for learning and experimentation. IREE continues from those compiler ideas into concrete deployment and runtime execution.

6 Compiler Stages You Should Recognize

You do not need to master every pass name on day one. You should, however, know what the major stages are for.

6.1 `flow`

The `flow` dialect models tensor program data flow and partitions work into dispatchable regions. A useful mental shortcut is:

> `flow` decides what chunks of tensor work should become executable dispatches.

6.2 `stream`

The `stream` dialect models placement, asynchronous execution, and resource lifetimes. This is where the program becomes much more explicitly scheduled.

> `stream` decides when work runs, where it runs, and how resources move through the schedule.

6.3 `hal`

The `hal` dialect represents operations against IREE's hardware abstraction layer.

> `hal` is where scheduling logic is expressed in a form that maps closely to the runtime API.

6.4 `vm`

The `vm` layer represents IREE virtual machine code and module structure. By the end of compilation, you are close to the deployable representation that the runtime will load.

7 Setting Up IREE

7.1 Build from Source

If you are studying IREE itself, building from source is the best path because it gives you the compiler tools, the runtime, and the source tree in one place.

7.1.1 Prerequisites

For a typical Linux setup, the official docs recommend:

- `cmake`,
- `ninja`,
- `clang`,
- `lld`.

On Debian or Ubuntu:

```
sudo apt install cmake ninja-build clang lld
```

Listing 2: Typical Linux prerequisites

7.1.2 Clone and Configure

```
git clone https://github.com/iree-org/iree.git
cd iree
git submodule update --init

cmake -G Ninja -B ../iree-build/ -S . \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DIREE_ENABLE_ASSERTIONS=ON \
  -DIREE_ENABLE_SPLIT_DWARF=ON \
  -DIREE_ENABLE_THIN_ARCHIVES=ON \
  -DCMAKE_C_COMPILER=clang \
  -DCMAKE_CXX_COMPILER=clang++ \
  -DIREE_ENABLE_LLD=ON
```

Listing 3: Clone and configure IREE

7.1.3 Build the Developer Tools

```
cmake --build ../iree-build/ \
  --target iree-compile iree-run-module iree-benchmark-module \
           iree-dump-module iree-opt iree-check-module
```

Listing 4: Build a minimal useful tool set

This course uses those tools constantly.

7.1.4 Run the Test Suite

```
./build_tools/cmake/ctest_all.sh ../iree-build
```

Listing 5: Run the CMake test wrapper

7.2 Python Packages

If you only need the packaged compiler and runtime, the current package names are:

- `iree-base-compiler`
- `iree-base-runtime`

The older package names `iree-compiler` and `iree-runtime` are deprecated.

```
python -m pip install iree-base-compiler iree-base-runtime
```

Listing 6: Install the packaged compiler and runtime

8 Meet the Main Tools

Tool	What it does
`iree-compile`	Compiles supported MLIR input into a deployable IREE module.
`iree-run-module`	Loads a compiled module and invokes an exported function.
`iree-benchmark-module`	Benchmarks module execution with the same style of inputs as `iree-run-module`.
`iree-dump-module`	Prints the contents of a `.vmfb` module.
`iree-opt`	Runs selected compiler passes and transformation pipelines on MLIR.
`iree-check-module`	Runs check-style tests embedded in compiled modules.
`iree-run-mlir`	Compiles and runs an MLIR file directly for testing and debugging.

9 How To Read an IREE Command Line

One reason IREE feels difficult at first is that the commands are dense. They become much easier once you read them as answers to a few fixed questions.

9.1 Reading a Compile Command

Consider this example:

```
iree-compile \
  --iree-hal-target-device=local \
  --iree-hal-local-target-device-backends=llvm-cpu \
  --iree-llvmcpu-target-cpu=host \
  simple_abs.mlir \
  -o simple_abs_cpu.vmfb
```

Listing 7: An annotated compile command

Read it like this:

1. **What is the input?** `simple_abs.mlir`

2. **What device family am I targeting?** `local`
3. **What executable backend do I want?** `llvm-cpu`
4. **How should CPU code be tuned?** `host`
5. **Where does the artifact go?** `simple_abs_cpu.vmfb`

9.2 Reading a Run Command

Now consider the matching runtime command:

```
iree-run-module \
  --module=simple_abs_cpu.vmfb \
  --device=local-task \
  --function=abs \
  --input=f32=-2
```

Listing 8: An annotated run command

Read it like this:

1. **What module am I loading?**
2. **What runtime device should execute it?**
3. **What exported function should be called?**
4. **What inputs should be passed?**

9.3 The Two Questions Beginners Mix Up

Do not confuse these:

- `-iree-hal-local-target-device-backends=llvm-cpu`: compile-time code generation choice.
- `-device=local-task`: runtime execution device choice.

The first changes what gets emitted. The second changes how the already-compiled module is executed.

9.4 A Tiny Backend Cheat Sheet

Backend	Good first use	Beginner note
`vmvx`	Reference execution and learning	Simple and portable, but not the fast path.
`llvm-cpu`	Real CPU deployment	Usually pair with `-iree-llvmcpu-target-cpu=host`.
`vulkan-spirv`	Vulkan-capable GPUs	Requires a working Vulkan runtime stack.
`cuda`	NVIDIA GPUs	Runtime device must also support CUDA.
`rocm` or `hip`	AMD GPUs	Platform-specific setup matters.

10 Your First IREE Program

We will start with a tiny MLIR program that computes an absolute value.

10.1 Step 1: Write `simple_abs.mlir`

```
module {
  func.func @abs(%arg0: tensor<f32>) -> tensor<f32> {
    %0 = math.absf %arg0 : tensor<f32>
    return %0 : tensor<f32>
  }
}
```

Listing 9: A minimal IREE-friendly MLIR example

10.2 Step 2: Compile It to `vmvx`

For a first run, `vmvx` is a nice backend because it is simple and portable. It is not the fast path, but it is a good learning path.

```
iree-compile \
  --iree-hal-target-device=local \
  --iree-hal-local-target-device-backends=vmvx \
  simple_abs.mlir \
  -o simple_abs_vmvx.vmfb
```

Listing 10: Compile the program to a VMFB using VMVX

10.3 Step 3: Run It

```
iree-run-module \
  --module=simple_abs_vmvx.vmfb \
  --device=local-task \
  --function=abs \
  --input=f32=-2
```

Listing 11: Execute the exported function

You should see a result equivalent to:

```
EXEC @abs
result[0]: hal.buffer_view
f32=2
```

Listing 12: Expected shape of the output

10.4 What Just Happened

1. `iree-compile` lowered your MLIR into IREE's internal pipeline.
2. It emitted a `.vmfb` module.
3. `iree-run-module` loaded that module.
4. The runtime selected the `local-task` CPU device.
5. The exported function `abs` was invoked with one scalar tensor input.

10.5 Shortcut for Experimentation

If you want a quick feedback loop while learning, you can skip the explicit `.vmfb` file and run the MLIR file directly:

```
iree-run-mlir \
  --iree-hal-target-device=local \
  --iree-hal-local-target-device-backends=vmvx \
  simple_abs.mlir \
  --function=abs \
  --input=f32=-2
```

Listing 13: Compile and run directly from MLIR

11 Running on CPU the Usual Way

For real CPU execution you will usually target `llvm-cpu`, not `vmvx`.

11.1 Compile for `llvm-cpu`

```
iree-compile \
  --iree-hal-target-device=local \
  --iree-hal-local-target-device-backends=llvm-cpu \
  --iree-llvmcpu-target-cpu=host \
  --iree-opt-level=O2 \
  simple_abs.mlir \
  -o simple_abs_cpu.vmfb
```

Listing 14: Compile for host CPU execution

Two flags matter here:

- `-iree-hal-local-target-device-backends=llvm-cpu`: use LLVM CPU code generation.
- `-iree-llvmcpu-target-cpu=host`: optimize for the machine you are compiling on.

If you omit the target CPU, IREE may fall back to a generic CPU target, which is easier to run everywhere but usually slower.

11.2 Run It

```
iree-run-module \
  --module=simple_abs_cpu.vmfb \
  --device=local-task \
  --function=abs \
  --input=f32=-2
```

Listing 15: Run the CPU-targeted module

11.3 Check Available Drivers and Devices

```
iree-run-module --list_drivers
iree-run-module --list_devices
```

Listing 16: Inspect what the runtime can see

For CPU work you should expect to see `local-sync://` and `local-task://`.

11.4 `local-sync` vs `local-task`

`local-sync` Synchronous, single-threaded, easier to follow in a debugger.

`local-task` Asynchronous, multithreaded, closer to typical production CPU execution.

When learning runtime internals, start with `local-sync`. When measuring ordinary CPU execution, use `local-task`.

12 Small Example 2: Scale a Vector

The first example used a scalar tensor. Now let us run something slightly more realistic.

12.1 Write `scale_by_two.mlir`

```
module {
  func.func @scale_by_two(%arg0: tensor<4xf32>) -> tensor<4xf32> {
    %cst = arith.constant dense<2.0> : tensor<4xf32>
    %0 = arith.mulf %arg0, %cst : tensor<4xf32>
    return %0 : tensor<4xf32>
  }
}
```

Listing 17: A tiny vector example

12.2 Compile and Run

```
iree-compile \
  --iree-hal-target-device=local \
  --iree-hal-local-target-device-backends=llvm-cpu \
  --iree-llvmcpu-target-cpu=host \
  scale_by_two.mlir \
  -o scale_by_two.vmfb

iree-run-module \
  --module=scale_by_two.vmfb \
  --device=local-task \
  --function=scale_by_two \
  --input="4xf32=1 -3 0.5 -2"
```

Listing 18: Compile and execute the vector example

Expected output:

```
EXEC @scale_by_two
result[0]: hal.buffer_view
4xf32=2 -6 1 -4
```

Listing 19: Expected output

12.3 Why This Example Matters

This small program introduces three useful habits:

- reading tensor input syntax,
- thinking about exported function signatures,
- separating the compiler target backend from the runtime device selection.

13 How `iree-run-module` Inputs Work

The input format is simple once you have seen a few examples.

MLIR type	Example flag
`i32`	`-input=1234`
`tensor<i32>`	`-input=i32=1234`
`tensor<2xi32>`	`-input="2xi32=12 34"`
`tensor<2x3xi32>`	`-input="2x3xi32=[1 2 3][4 5 6]"`

13.1 Practical Advice

- Quote inputs that contain spaces.
- Match the function signature exactly.
- If you are unsure what the function expects, inspect the source MLIR or dump the compiled module.
- Inputs can also be loaded from raw binary files or NumPy `.npy` files.

14 Learning the Pipeline by Dumping Phases

One of the best beginner techniques in IREE is to dump the IR after each compiler phase and compare the files.

14.1 Dump All Compilation Phases

```
mkdir -p /tmp/iree/simple_abs

iree-compile simple_abs.mlir \
  --iree-hal-target-device=local \
  --iree-hal-local-target-device-backends=llvm-cpu \
  --iree-llvmcpu-target-cpu=host \
  --dump-compilation-phases-to=/tmp/iree/simple_abs \
  -o /tmp/iree/simple_abs/simple_abs_cpu.vmfb
```

Listing 20: Dump the pipeline step by step

You should get files with names shaped like:

```
simple_abs.1.input.mlir
simple_abs.2.abi.mlir
simple_abs.3.preprocessing.mlir
simple_abs.4.global-optimization.mlir
simple_abs.5.dispatch-creation.mlir
simple_abs.6.flow.mlir
simple_abs.7.stream.mlir
simple_abs.8.executable-sources.mlir
simple_abs.9.executable-configurations.mlir
simple_abs.10.executable-targets.mlir
simple_abs.11.hal.mlir
simple_abs.12.vm.mlir
```

Listing 21: Typical dumped phase files

14.2 How To Read Them

`input` Your starting point.

`abi` Entry-point and calling-convention shaping.

`preprocessing` Canonical cleanup and early normalization.

`global-optimization` Whole-program simplifications.

`dispatch-creation` Compute regions become explicit dispatch candidates.

`flow` Tensor work is organized as dispatchable data flow.

`stream` Scheduling and resource management become more explicit.

`hal` The program is expressed in terms of device-facing runtime concepts.

`vm` The final VM-oriented representation before packaging.

14.3 Why This Is So Valuable

If you come from xDSL or general MLIR learning, this is the closest thing to a live anatomy lesson for IREE. Instead of memorizing names, you can watch the same program become progressively lower level.

15 HAL for Beginners

HAL is one of the most important concepts in IREE, and it is also one of the most misunderstood.

15.1 What HAL Is

HAL stands for Hardware Abstraction Layer. It is the layer between the runtime and actual hardware-specific implementations.

HAL exists in two related forms:

- a **compile-time MLIR dialect** named `hal`,
- a **runtime API and driver interface** implemented in the runtime.

15.2 Compile-Time HAL

At compile time, HAL is used to express operations such as:

- device interaction,
- buffer management,
- executable packaging,
- synchronization and dispatch structure.

The important beginner point is this:

> IREE does not use HAL to represent the math itself. It uses HAL to represent how compiled work is prepared and executed on devices.

15.3 Runtime HAL

At runtime, HAL drivers implement the abstract operations required by the compiler output. That includes things like:

- buffer allocation,
- command submission,
- fences and semaphores,
- loading executable payloads,
- device enumeration.

15.4 Target Backend vs Runtime Driver

This distinction matters:

- **Target backend**: compile-time code generation choice.
- **Runtime driver**: execution-time device interface.

Examples:

- `llvm-cpu` is a compiler backend.
- `local-task` and `local-sync` are runtime drivers/devices for CPU execution.
- `vulkan-spirv` is a compiler backend.
- `vulkan` is the runtime driver.

If you mix these up, many IREE command lines stop making sense.

16 Runtime Building Blocks

Once you move past the first few CLI examples, the runtime becomes much easier to understand if you know its core objects.

16.1 Driver

The driver is the top-level runtime entry point for a device family. It is responsible for discovering devices and creating device instances.

> Think of the driver as the runtime-side bridge between "I know I want Vulkan or local CPU" and "give me an actual device object I can use".

16.2 Device

The device is the execution surface. Command buffers are submitted to devices, buffers are allocated through device-associated allocators, and synchronization is observed at this level.

For beginners, the most visible devices are:

- `local-sync`
- `local-task`
- GPU-facing devices such as `vulkan` or `cuda`

16.3 Allocator and Buffers

Tensor data eventually lives in buffers managed by HAL allocators. A buffer may live in host-visible memory, device-local memory, or some combination depending on the backend.

This matters because a lot of "why is this runtime code so large?" questions reduce to:

- where does the data live,
- who is allowed to see it,
- when is movement or mapping required.

16.4 Command Buffer

A command buffer records work to be executed later. In IREE this includes dispatches, transfers, and synchronization-related operations.

If you have seen Vulkan before, this concept should feel familiar. If not, use this shortcut:

> A command buffer is a recorded plan for device work, not the work itself.

16.5 Executable

An executable is the compiled device-facing payload. It may come from LLVM CPU code generation, SPIR-V, CUDA-related lowering, or other backend-specific formats.

The runtime does not "re-derive" the math from scratch. It loads executable payloads that the compiler already produced.

16.6 Semaphores and Fences

These are synchronization tools:

- **semaphores** coordinate ordering across submissions and device work,
- **fences** help coordinate between host and device completion points.

You do not need to implement them to benefit from them. You only need to understand that correct execution on asynchronous systems requires explicit ordering.

16.7 Why This Section Matters

When you later read files under `runtime/src/iree/hal/`, you will keep seeing the same nouns: driver, device, allocator, command buffer, executable, semaphore. Learning those nouns early makes the runtime code much less intimidating.

17 VMFB and IREE Modules

17.1 What a VMFB Is

A `.vmfb` file is an IREE module packaged in FlatBuffer form. It typically contains:

- VM bytecode and metadata,
- references to required runtime modules,
- executable payloads for one or more targets,
- exported function information.

17.2 Inspect a Module

```
iree-dump-module simple_abs_cpu.vmfb
```

Listing 22: Inspect a VMFB

You will see information such as:

- required types,
- module dependencies,
- imported functions,
- exported functions.

This is a very useful habit when you are unsure whether a function is really exported or what runtime modules the artifact expects.

17.3 A Useful Mental Model

Many beginners imagine that a program is just one blob of machine code. In IREE, it is better to think in terms of **modules plus executables plus runtime services**. The VM coordinates modules. The HAL handles device-facing work. The executable payloads do the dense compute.

18 Small How-To 1: Benchmark a Module

After you can compile and run a program once, the next useful step is to benchmark it.

```
iree-benchmark-module \
  --module=simple_abs_cpu.vmfb \
  --device=local-task \
  --function=abs \
  --input=f32=-2
```

Listing 23: Benchmark the same exported function

This measures full module invocation overhead, including VM and runtime activity, not just one raw kernel in isolation.

19 Small How-To 2: Dump Executable Files

If you want to see what device-specific code was generated, ask the compiler to dump executable-related files.

```
mkdir -p /tmp/iree/simple_abs_dump

iree-compile simple_abs.mlir \
  --iree-hal-target-device=local \
  --iree-hal-local-target-device-backends=llvm-cpu \
  --iree-llvmcpu-link-embedded=false \
  --iree-hal-dump-executable-files-to=/tmp/iree/simple_abs_dump \
  -o /tmp/iree/simple_abs_dump/simple_abs_cpu.vmfb
```

Listing 24: Dump generated executable artifacts

You may see files such as:

- `module_abs_dispatch_0.mlir`,

- `module_abs_dispatch_0_system_elf_x86_64.codegen.ll`,
- `module_abs_dispatch_0_system_elf_x86_64.o`,
- `module_abs_dispatch_0_system_elf_x86_64.so`.

This is where the compiler stops feeling abstract. You can literally inspect the generated dispatch sources and target-specific binaries.

20 Small How-To 3: Isolate a Dispatch Benchmark

The compiler can also dump benchmarkable dispatch modules.

```
iree-compile simple_abs.mlir \
  --iree-hal-target-device=local \
  --iree-hal-local-target-device-backends=llvm-cpu \
  --iree-hal-dump-executable-benchmarks-to=/tmp/iree/simple_abs_bench \
  -o /dev/null

iree-compile \
  /tmp/iree/simple_abs_bench/
    module_abs_dispatch_0_embedded_elf_x86_64_benchmark.mlir \
  -o /tmp/iree/simple_abs_bench/module_abs_dispatch_0_benchmark.vmfb

iree-benchmark-module \
  --module=/tmp/iree/simple_abs_bench/module_abs_dispatch_0_benchmark.
    vmfb
```

Listing 25: Generate and benchmark a single dispatch module

This is useful when you want to understand the performance of one dispatch without all surrounding application logic. These generated benchmark modules use fake buffers and synthetic operands, so treat them as a starting point for investigation, not as a substitute for a real end-to-end benchmark.

21 Small How-To 4: Import an ONNX Model

For a beginner course it is enough to understand the shape of the workflow.

```
python -m pip install "iree-base-compiler[onnx]" iree-base-runtime

iree-import-onnx mobilenetv2-10.onnx --opset-version 17 -o mobilenetv2.
    mlir

iree-compile \
  --iree-hal-target-device=local \
  --iree-hal-local-target-device-backends=llvm-cpu \
  --iree-llvmcpu-target-cpu=host \
  --iree-opt-level=O2 \
  mobilenetv2.mlir \
  -o mobilenet_cpu.vmfb

iree-run-module \
  --module=mobilenet_cpu.vmfb \
  --device=local-task \
  --function=... \
  --input="1x3x224x224xf32=0"
```

Listing 26: Import ONNX

The last two placeholders are model-specific:

- the exported function name,
- the exact input signature.

If you are working from PyTorch instead of ONNX, the official PyTorch frontend is `iree-turbine`.

22 Debugging

22.1 Use `local-sync` First

When stepping through runtime execution, prefer `local-sync` because it avoids multithreaded scheduling noise.

22.2 Debug `iree-run-module` with LLDB

```
lldb -- ./iree-build/tools/iree-run-module \
  --module=simple_abs_cpu.vmfb \
  --device=local-sync \
  --function=abs \
  --input=f32=-2
```

Listing 27: Launch the runtime under LLDB

Useful LLDB commands:

- `break set -n function_name`
- `break set -r iree_hal.*`
- `run`
- `next`
- `step`
- `continue`
- `bt`
- `frame variable`

22.3 GDB Alternative

```
gdb --args ./iree-build/tools/iree-run-module \
  --module=simple_abs_cpu.vmfb \
  --device=local-sync \
  --function=abs \
  --input=f32=-2
```

Listing 28: Launch the runtime under GDB

22.4 Three Good Debug Habits

1. Dump the module with `iree-dump-module`.
2. Dump compiler phases with `-dump-compilation-phases-to`.
3. Switch from `local-task` to `local-sync` when you need a clearer execution story.

23 Troubleshooting Checklist

When something fails in IREE, the fastest path is usually to classify the failure before changing random flags.

"Device not found" or "driver not available" Run `iree-run-module -list_drivers` and `-list_devices`. Confirm that your runtime was built with the driver you expect.

"Function not found" Use `iree-dump-module` and verify the exported function name.

Shape or type mismatch Compare your `-input=` flags against the function signature in the source MLIR or dumped module.

CPU result is correct but unexpectedly slow Rebuild or recompile with `-iree-llvmcpu-target-cpu=ho` Also make sure you are not benchmarking `vmvx` by accident.

Execution is hard to follow Switch from `local-task` to `local-sync` and rerun under a debugger.

Compiler pipeline feels opaque Use `-dump-compilation-phases-to=` and compare `flow`, `stream`, `hal`, and `vm`.

23.1 A Good Investigation Order

For most beginner issues, this order works well:

1. verify the source MLIR,
2. compile with explicit backend flags,
3. dump the module,
4. run with explicit device selection,
5. switch to `local-sync` if needed,
6. only then start reading runtime internals.

24 Common Beginner Mistakes

1. **Confusing backend and device.** `llvm-cpu` is not a runtime device string; `local-task` is.
2. **Forgetting to export or call the right function.** The runtime only sees exported functions.
3. **Passing the wrong input shape.** Match the exact tensor shape and element type.
4. **Using `vmvx` and expecting top CPU performance.** `vmvx` is primarily a reference backend.
5. **Ignoring the target CPU flag.** On CPU, `-iree-llvmcpu-target-cpu=host` is often the right default when not cross-compiling.
6. **Trying to learn everything at once.** First learn compile, run, inspect, and benchmark. Then read the lower-level runtime code.

25 A Beginner Reading Order for the Source Tree

Once you are ready to inspect the IREE repository itself, this is a practical order:

1. `samples/` to see small end-to-end examples,
2. `tools/` to understand the CLI front doors,
3. compiler dialect docs for `flow`, `stream`, `hal`, and `vm`,
4. `runtime/src/iree/hal/` for the HAL API and drivers,
5. the CPU runtime drivers such as `local-sync` and `local-task`.

26 Glossary

ABI Application Binary Interface. In IREE, this includes how entry points are shaped and called.

Bytecode module A module containing VM bytecode interpreted by IREE's virtual machine.

Dispatch A unit of scheduled executable work.

Executable Target-specific compiled compute payload attached to a module.

HAL Hardware Abstraction Layer used by the compiler and runtime.

Importer A tool that converts a framework-specific program into a form IREE can compile.

MLIR The compiler infrastructure beneath IREE.

Runtime driver The execution-time implementation for a device family.

Stream IREE dialect handling placement, scheduling, and resource lifetime modeling.

Target backend The compile-time code generation backend.

VM IREE's virtual machine layer.

VMFB The deployable FlatBuffer container produced by the compiler.

VMVX A portable reference executable backend used frequently in examples and testing.

27 Exercises

Try these in order:

1. Compile `simple_abs.mlir` to both `vmvx` and `llvm-cpu`. Compare the commands and outputs.
2. Dump the compilation phases for `simple_abs.mlir`. Open `simple_abs.6.flow.mlir` and `simple_abs.7.stream.mlir`. Write down the biggest structural difference you notice.
3. Modify `scale_by_two.mlir` so it multiplies by `3.0` instead of `2.0`.
4. Run the same module once with `local-sync` and once with `local-task`.
5. Use `iree-dump-module` on a compiled VMFB and identify the exported function names.
6. Import a tiny ONNX model and compile it for CPU.

28 Where To Go Next

After this course, a good next step is one of these:

- learn one frontend path in depth, such as ONNX or PyTorch through `iree-turbine`,
- study the `flow -> stream -> hal` lowering path by diffing dumped phases,
- read the CPU deployment and benchmarking docs and start measuring real workloads,
- inspect the runtime HAL API and one concrete driver implementation.

29 Official References

These are the main official references that informed the practical commands in this course:

- Getting Started
- Developer Overview
- CPU Deployment Guide
- Benchmarking
- Developer Tips and Tricks
- IREE MLIR Dialects Reference

30 Closing Note

If you remember only five things after the first pass through this material, remember these:

1. IREE is both a compiler and a runtime.
2. `flow`, `stream`, `hal`, and `vm` are the key pipeline names to recognize.
3. `llvm-cpu` is a compile-time backend, while `local-task` and `local-sync` are runtime devices.
4. `.vmfb` is the deployable module format you run.
5. The fastest way to learn IREE is to compile, dump, inspect, and rerun tiny examples.